INTRODUCTION

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Distributed Systems
- Special-Purpose Systems
- Computing Environments
- Open-Source Operating Systems

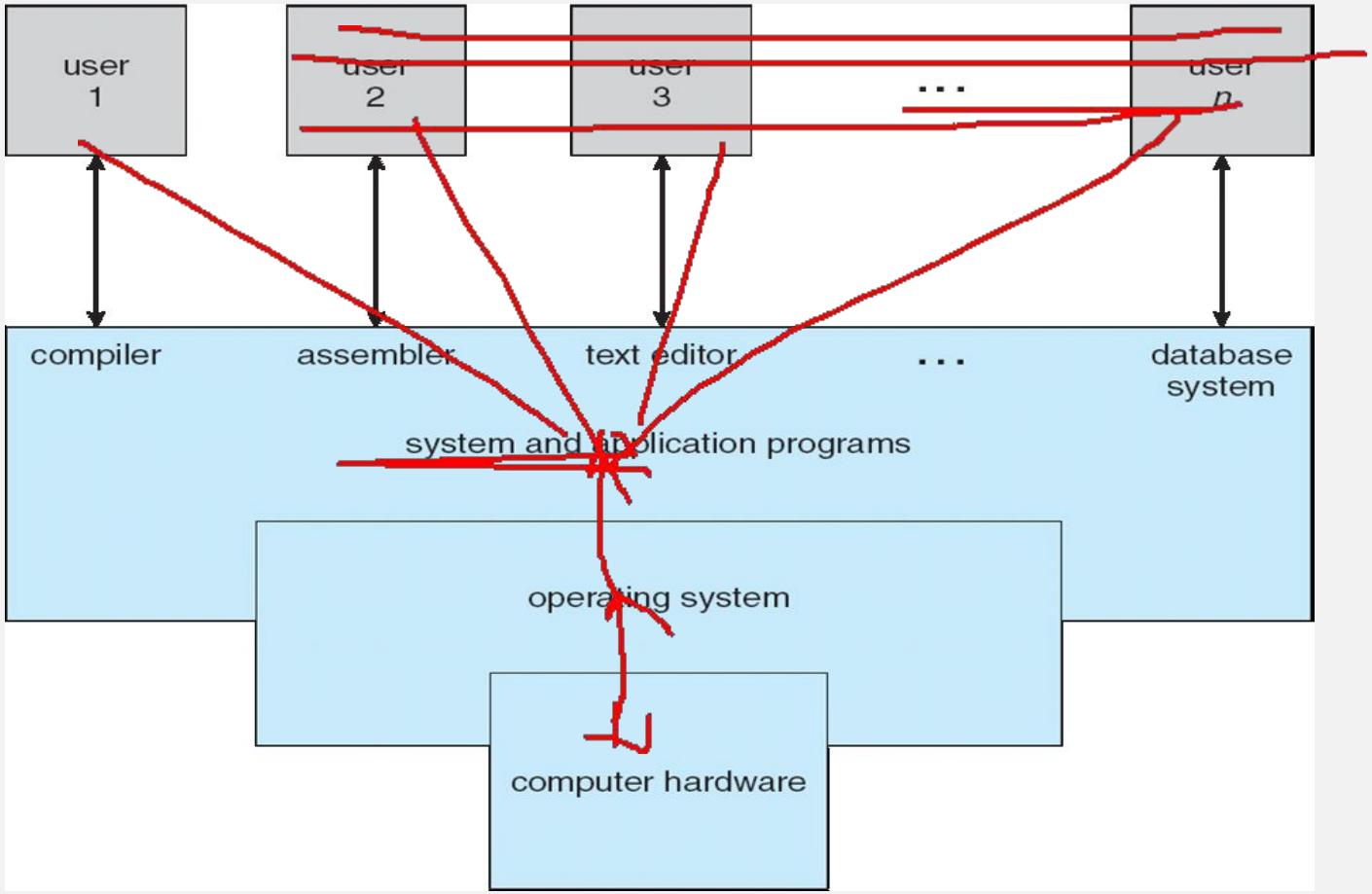
WHAT IS AN OPERATING SYSTEM?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

COMPUTER SYSTEM STRUCTURE

- Computer system can be divided into four components
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

FOUR COMPONENTS OF A COMPUTER SYSTEM



OPERATING SYSTEM DEFINITION

- A **compiler** is a Computer program that translate computer code written in one programming language (the *source* language) into another language (the *target* language). The name "compiler" is primarily used for programs that translate source code from a high level language to a low level programming language (e.g. assembly language, object code) to create an executable program.
- **An assembler** is a computer program that translates assembly language code into machine code. This allows a computer to directly communicate with its hardware

•The assembler takes in human-readable assembly language instructions
•It translates the instructions into binary code, or machine code
•The computer's central processing unit (CPU) can then execute the machine code

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer

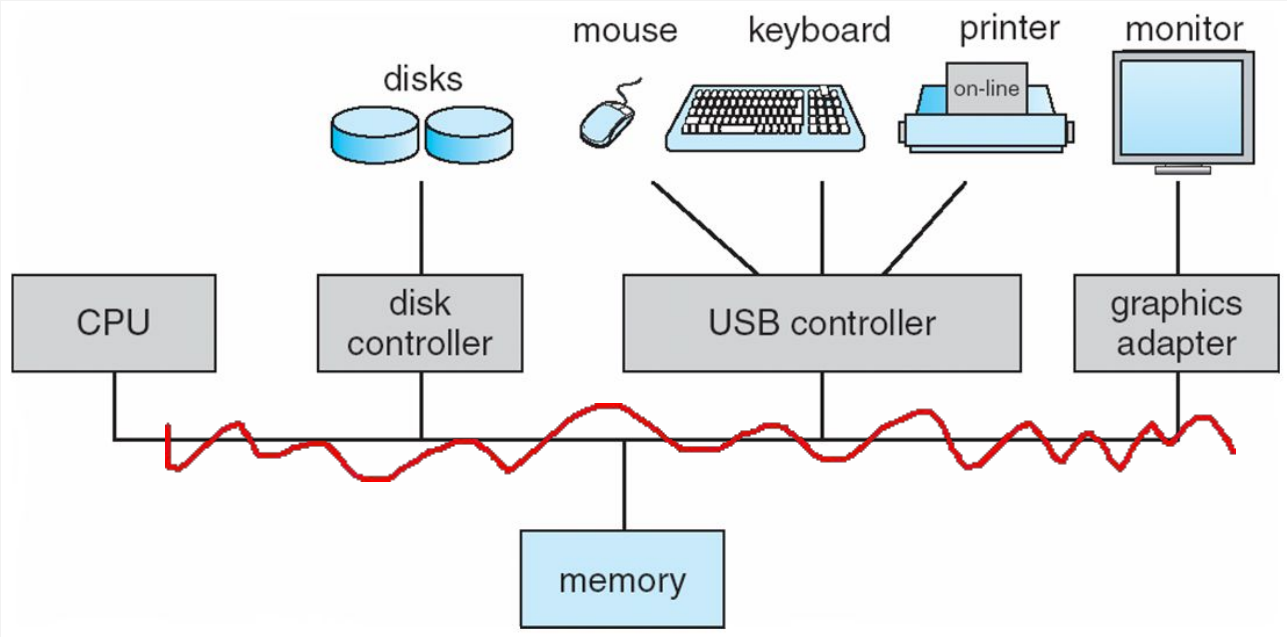
OPERATING SYSTEM DEFINITION (CONT)

- A **text editor** is a type of Computer Program that edits plain text. An example of such program is "**notepad**" software (e.g. Windows Notepads). Text editors are provided with Operating System and software development packages, and can be used to change files such as Configuration files, documentation files and Programming language source code.

- No universally accepted definition
- " "The one program running at all times on the computer" is the **kernel.** Everything else is either a system program (ships with the operating system) or an application program.

COMPUTER SYSTEM ORGANIZATION…

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



COMPUTER STARTUP

- **bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EPROM, generally known as **firmware**
  - Initializes all aspects of system including the CPU registers, device controllers, and memory contents
  - Locates and loads operating system kernel and starts execution of the first process (such as "init") and waits for events to occur.

COMMON FUNCTIONS OF INTERRUPTS

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- Incoming interrupts are *disabled* while another interrupt is being processed to prevent a *lost interrupt*
- A *trap* is a software-generated interrupt caused either by an error such as divide by 0 or a user request (system call).
- An operating system is **interrupt driven**

INTERRUPT HANDLING

- The operating system preserves the state of the CPU by storing registers and the program counter
- Separate segments of code determine what action should be taken for each type of interrupt

I/O STRUCTURE

- After I/O starts, control returns to user program without waiting for I/O completion
  - **System call** – request to the operating system to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state
  - Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt

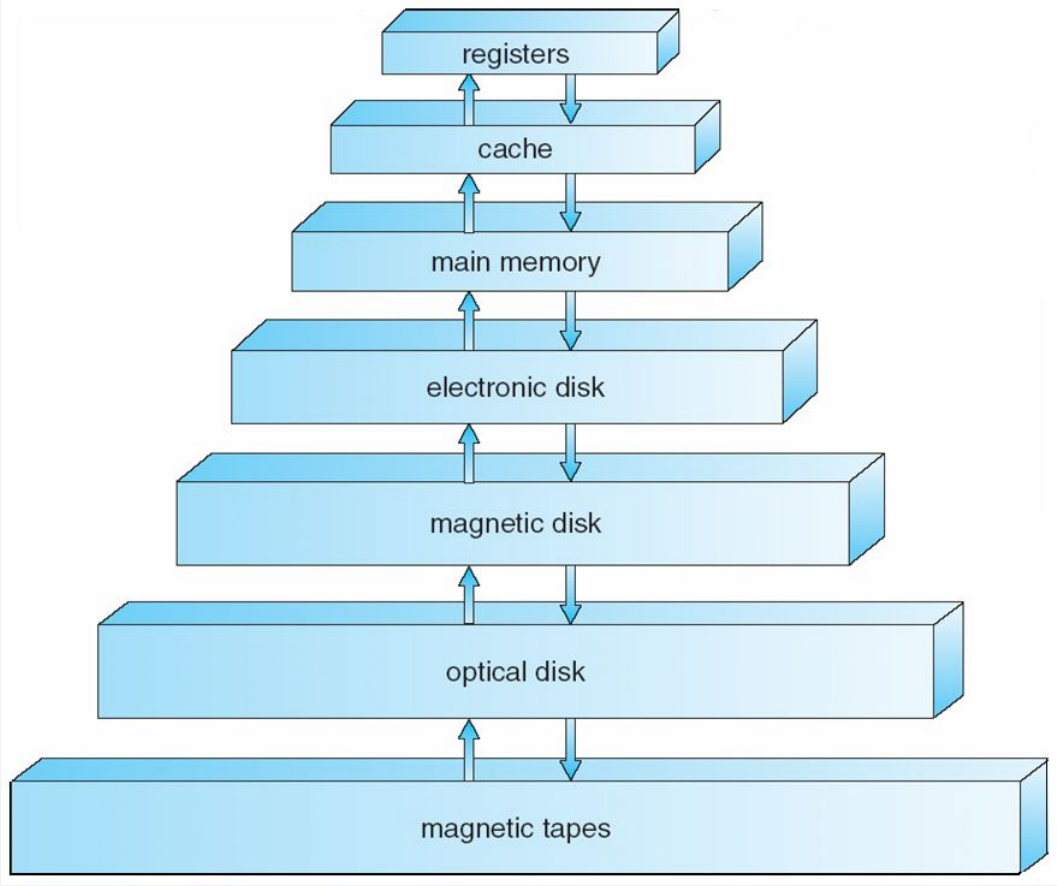
STORAGE STRUCTURE

- Main memory – only large storage media that the CPU can access directly
- Programs and data cannot reside in main memory permanently because:
  - Main memory is limited (too small) to store all programs and data permanently
  - Main memory is volatile
- So secondary storage is provided – extension of main memory that provides large nonvolatile storage capacity
- Magnetic disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer

STORAGE HIERARCHY

- Storage systems organized in hierarchy by
  - Speed
  - Cost
  - Volatility
- The higher levels in the hierarchy are expensive but fast. As we move down the hierarchy, the cost per bit generally decreases, whereas the access time generally increases.
- **Caching** – copying information into faster storage system; main memory can be viewed as a last *cache* for secondary storage

STORAGE-DEVICE HIERARCHY



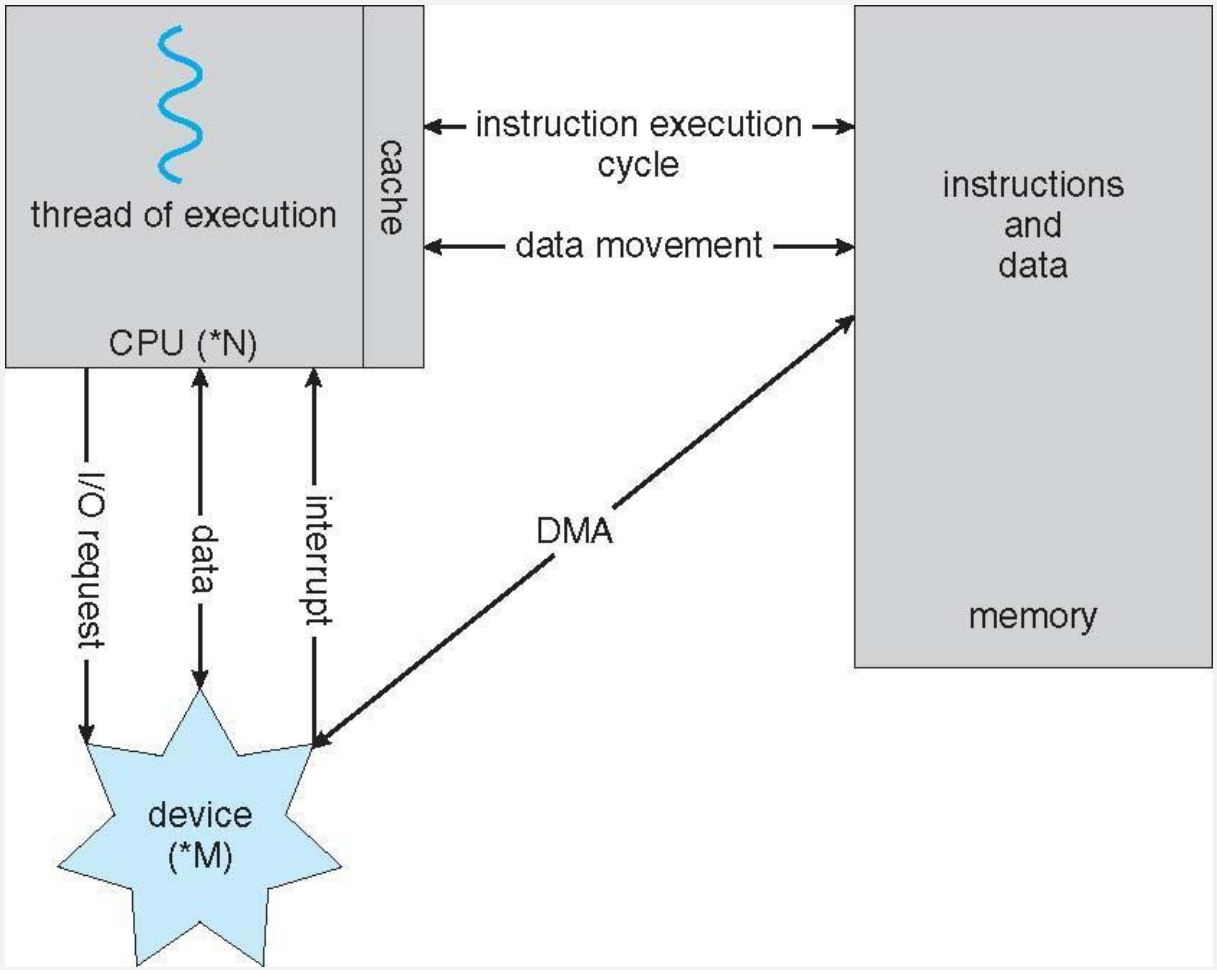
CACHING

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem
  - Cache size and replacement policy

DIRECT MEMORY ACCESS STRUCTURE

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte

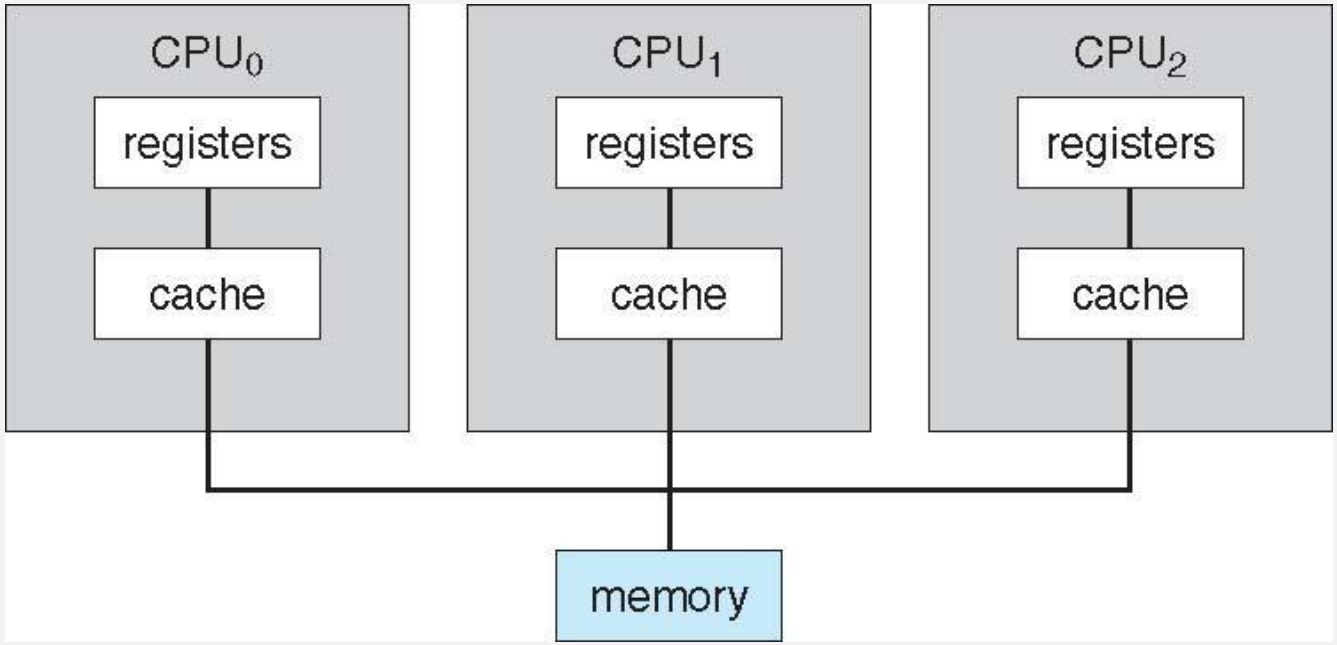
HOW A MODERN COMPUTER WORKS



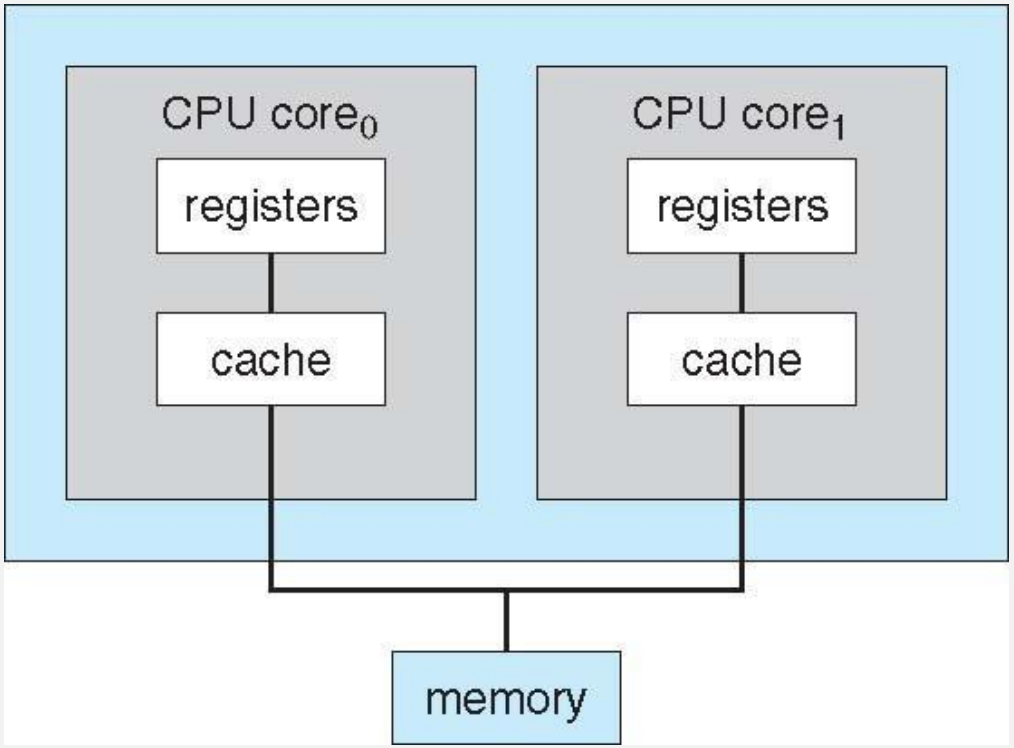
COMPUTER-SYSTEM ARCHITECTURE

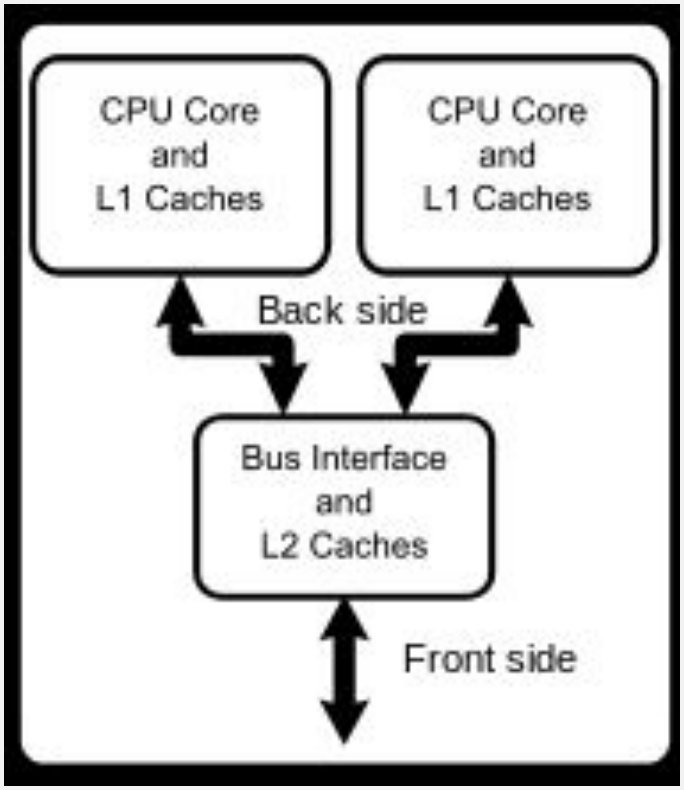
- Most systems use a single general-purpose processor
  - Most systems have special-purpose processors as well
- Multiprocessors systems growing in use and importance
  - Also known as parallel systems, tightly-coupled systems
  - Advantages include
    1. Increased throughput
    2. Economy of scale
    3. Increased reliability – graceful degradation or fault tolerance
  - Two types
    1. Asymmetric Multiprocessing
    2. Symmetric Multiprocessing

SYMMETRIC MULTIPROCESSING ARCHITECTURE



A DUAL-CORE DESIGN





DISTRIBUTED SYSTEMS

- Computation is distributed among several processors. In contrast to tightly-coupled systems, the processors do not share a clock or memory. Each has its own local memory.

- Communication is via a network. These systems are termed loosely-coupled or distributed systems. The processors vary in size and function and are called nodes.

- Advantages of distributed systems:
  - **Reliability**: If one node fails, the remaining nodes can continue operating. So by building enough redundancy, the system will not fail if one or more nodes fail (e.g. redundant web servers).

  - **Computation speedup**: Computation can be distributed among various nodes to run concurrently (e.g. load balanced web servers).

DISTRIBUTED SYSTEMS (CONTD.)

- **Resource Sharing:** Software, data, and hardware resources can be shared. E.g. data files in node A can be accessed by a user at node B. Files can be printed at a shared laser printer.

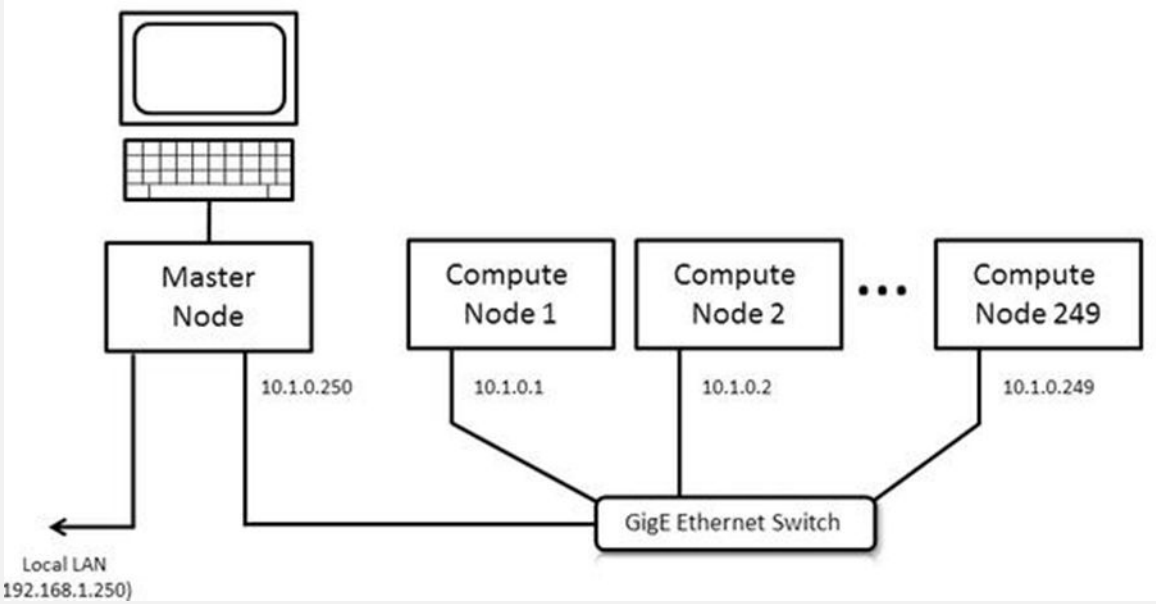
- **Communication:** Processes at various nodes can exchange information.

CLUSTERED SYSTEMS

- Are a form of distributed systems. Composed of 2 or more independent machines coupled together.
  - Usually sharing storage via a storage-area network (SAN)
  - Provides a high-availability service which survives failures
    - Asymmetric clustering has one machine in hot-standby mode while other machine/server run applications. The hot-standby machine only monitors the active server. If it fails, the hot-standby machine becomes the active server.
    - Symmetric clustering has multiple nodes running applications, monitoring each other
  - Some clusters are for high-performance computing (HPC)
    - Applications must be written to use parallelization

CLUSTERED SYSTEMS (CONTD.)

- An example is the Beowulf cluster where the cluster master node does the management and provisioning and a set of compute nodes that do computations.



- Beowulf Clusters are scalable performance clusters based on commodity hardware, on a private network, with open source software (Linux) infrastructure. Each consists of a cluster of PCs or workstations dedicated to running high-performance computing tasks.

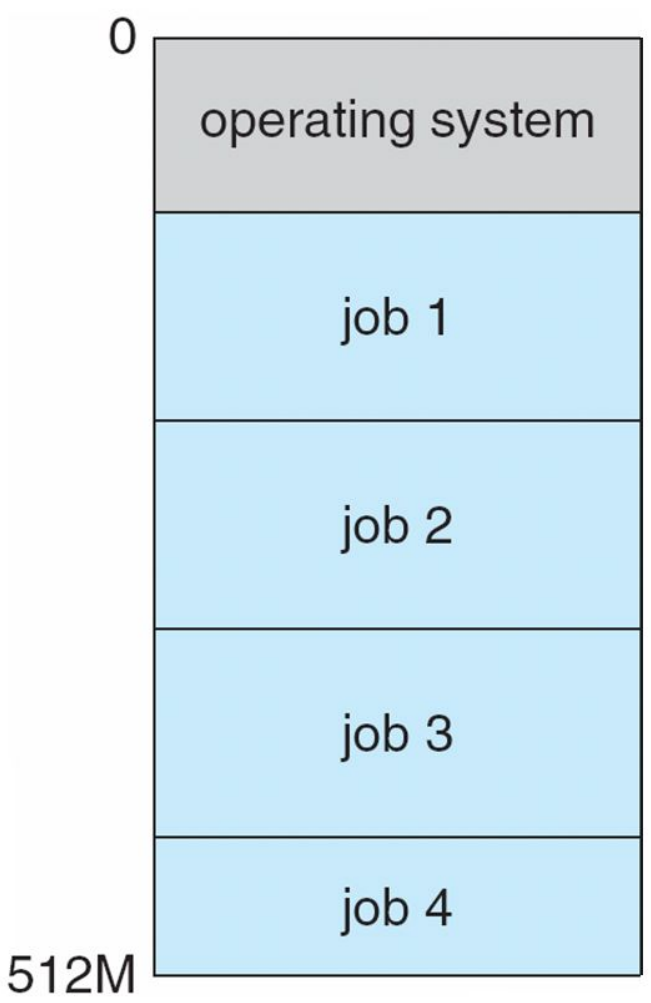
OPERATING SYSTEM STRUCTURE

- **Multiprogramming** needed for efficiency
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
  - When it has to wait (for I/O for example), OS switches to another job

OPERATING SYSTEM STRUCTURE (CONT.)

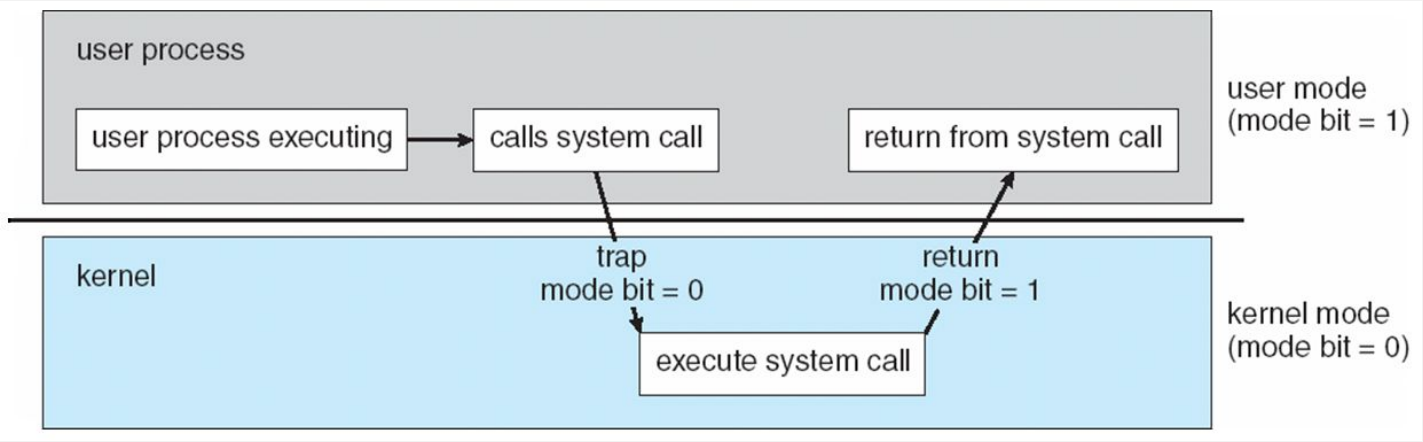
- **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory □ **process**
  - If several jobs ready to run at the same time □ **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run
  - **Virtual memory** allows execution of processes not completely in memory

MEMORY LAYOUT FOR MULTIPROGRAMMED SYSTEM



TRANSITION FROM USER TO KERNEL MODE

- Timer to prevent infinite loop / process hogging resources
  - Set interrupt after specific period
  - Operating system decrements counter
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time



PROCESS MANAGEMENT

- A process is a program in execution. It is a unit of work within the system. Program is a *passive entity*, process is an *active entity*.
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
  - Concurrency by multiplexing the CPUs among the processes / threads

PROCESS MANAGEMENT ACTIVITIES

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

MEMORY MANAGEMENT

- All data in memory before and after processing
- All instructions required in memory in order to execute
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

STORAGE MANAGEMENT

- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
  - Each medium is controlled by device (i.e., disk drive, tape drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

MASS-STORAGE MANAGEMENT

- Usually disks used to store data that does not fit in main memory or data that must be kept for a "long" period of time
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
  - Free-space management
  - Storage allocation
  - Disk scheduling
- Some storage need not be fast
  - Tertiary storage includes optical storage, magnetic tape
  - Varies between WORM (write-once, read-many-times) and RW (read-write)

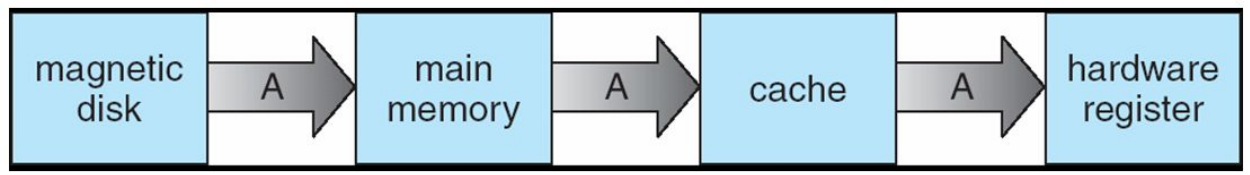
PERFORMANCE OF VARIOUS LEVELS OF STORAGE

- Movement between levels of storage hierarchy can be explicit or implicit

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

MIGRATION OF INTEGER A FROM DISK TO REGISTER

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide cache coherency in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
  - Several copies of a datum can exist

PROTECTION AND SECURITY

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS.

- It means being able to distinguish between authorized and unauthorized access. E.g. userid/password, smart cards with biometrics data etc.

- A system can have adequate protection (authorization) and still be prone to failure and inappropriate access (e.g. if a user's userid/password is stolen). Then data can be copied or deleted.

PROTECTION AND SECURITY

- **Security** – It is the job of security to defend a system against internal and external attacks.
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service

- While defending against some of these attacks may be considered to be an O/S function on some systems, it is left to specialized software on other systems (viruses, malware etc.) such as Norton's Security Suite.

PROTECTION AND SECURITY

Most O/S' maintain UIDs and GIDs to generally first distinguish among users, to determine who can do what:

- User identities (**user IDs**, security IDs) include name and associated number, one per user
- User ID then associated with all files, processes of that user to determine access control
- Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file. E.g. in UNIX a file owner may perform all operations on a file while the group members may only be allowed to read a file.
- **Privilege escalation** allows user to change to effective ID with more rights

OPERATING SYSTEM SERVICES

OS SERVICES

USERS

User Interface

Program Execution

I/O Operations

File-System Manipulation

Communication

Error Detection

SYSTEM PROGRAMS
For ensuring efficiency of OS

Resource Allocation

Accounting

Protection and Security

Services of Operating System

- Program execution
- Input Output Operations
- Communication between Process
- File Management
- Memory Management
- Process Management
- Security and Privacy
- Resource Management
- User Interface
- Networking
- Error handling
- Time Management

Program Execution

It is the Operating System that manages how a program is going to be executed. It loads the program into the memory after which it is executed. The order in which they are executed depends on the CPU Scheduling Algorithms. A few are FCFS, SJF, etc. When the program is in execution, the Operating System also handles deadlock i.e. no two processes come for execution at the same time. The Operating System is responsible for the smooth execution of both user and system programs. The Operating System utilizes various resources available for the efficient running of all types of functionalities.

Input Output Operations

Operating System manages the input-output operations and establishes communication between the user and device drivers. Device drivers are software that is associated with hardware that is being managed by the OS so that the sync between the devices works properly. It also provides access to input-output devices to a program when needed.

Communication Between Processes

The Operating system manages the communication between processes. Communication between processes includes data transfer among them. If the processes are not on the same computer but connected through a computer network, then also their communication is managed by the Operating System itself.

File Management

The operating system helps in managing files also. If a program needs access to a file, it is the operating system that grants access. These permissions include read-only, read-write, etc. It also provides a platform for the user to create, and delete files. The Operating System is responsible for making decisions regarding the storage of all types of data or files, i.e, floppy disk/hard disk/pen drive, etc. The Operating System decides how the data should be manipulated and stored.

Memory Management

Let's understand memory management by OS in simple way. Imagine a cricket team with limited number of player . The team manager (OS) decide whether the upcoming player will be in playing 11 ,playing 15 or will not be included in team , based on his performance . In the same way, OS first check whether the upcoming program fulfil all requirement to get memory space or not ,if all things good, it checks how much memory space will be sufficient for program and then load the program into memory at certain location. And thus , it prevents program from using unnecessary memory.

Process Management

Let's understand the process management in unique way. Imagine, our kitchen stove as the (CPU) where all cooking(execution) is really happen and chef as the (OS) who uses kitchen-stove(CPU) to cook different dishes(program). The chef(OS) has to cook different dishes(programs) so he ensure that any particular dish(program) does not take long time(unnecessary time) and all dishes(programs) gets a chance to cooked(execution) .The chef(OS) basically scheduled time for all dishes(programs) to run kitchen(all the system) smoothly and thus cooked(execute) all the different dishes(programs) efficiently.

Security and Privacy

Security : OS keep our computer safe from an unauthorized user by adding security layer to it. Basically, Security is nothing but just a layer of protection which protect computer from bad guys like viruses and hackers. OS provide us defenses like firewalls and anti-virus software and ensure good safety of computer and personal information.

Privacy : OS give us facility to keep our essential information hidden like having a lock on our door, where only you can enter and other are not allowed . Basically , it respect our secrets and provide us facility to keep it safe.

Resource Management

System resources are shared between various processes. It is the Operating system that manages resource sharing. It also manages the CPU time among processes using CPU Scheduling Algorithms. It also helps in the memory management of the system. It also controls input-output devices. The OS also ensures the proper use of all the resources available by deciding which resource to be used by whom.

Networking

This service enables communication between devices on a network, such as connecting to the internet, sending and receiving data packets, and managing network connections.

User Interface

User interface is essential and all operating systems provide it. Users either interacts with the operating system through the command-line interface or graphical user interface or GUI. The command interpreter executes the next user-specified command.
A GUI offers the user a mouse-based window and menu system as an interface.

Error Handling

The Operating System also handles the error occurring in the CPU, in Input-Output devices, etc. It also ensures that an error does not occur frequently and fixes the errors. It also prevents the process from coming to a deadlock. It also looks for any type of error or bugs that can occur while any task. The well-secured OS sometimes also acts as a countermeasure for preventing any sort of breach of the Computer System from any external source and probably handling them.

Time Management

Imagine traffic light as (OS), which indicates all the cars(programs) whether it should be stop(red)=>(simple queue), start(yellow)=>(ready queue),move(green)=>(under execution) and this light (control) changes after a certain interval of time at each side of the road(computer system) so that the cars(program) from all side of road move smoothly without traffic.

System Calls

System calls are interfaces provisioned by the operating system to allow user-level applications to interact with low-level hardware components & make use of all the services provided by the kernel, which is a core component and the heart of an operating system that manages all the hardware and the services provided by the OS.

These system calls are essential for every process to interact with the kernel and properly use the services provided by it. System calls are an interface between a process and the operating system. And they're the only way to switch from user mode to kernel mode.

Types of System Calls

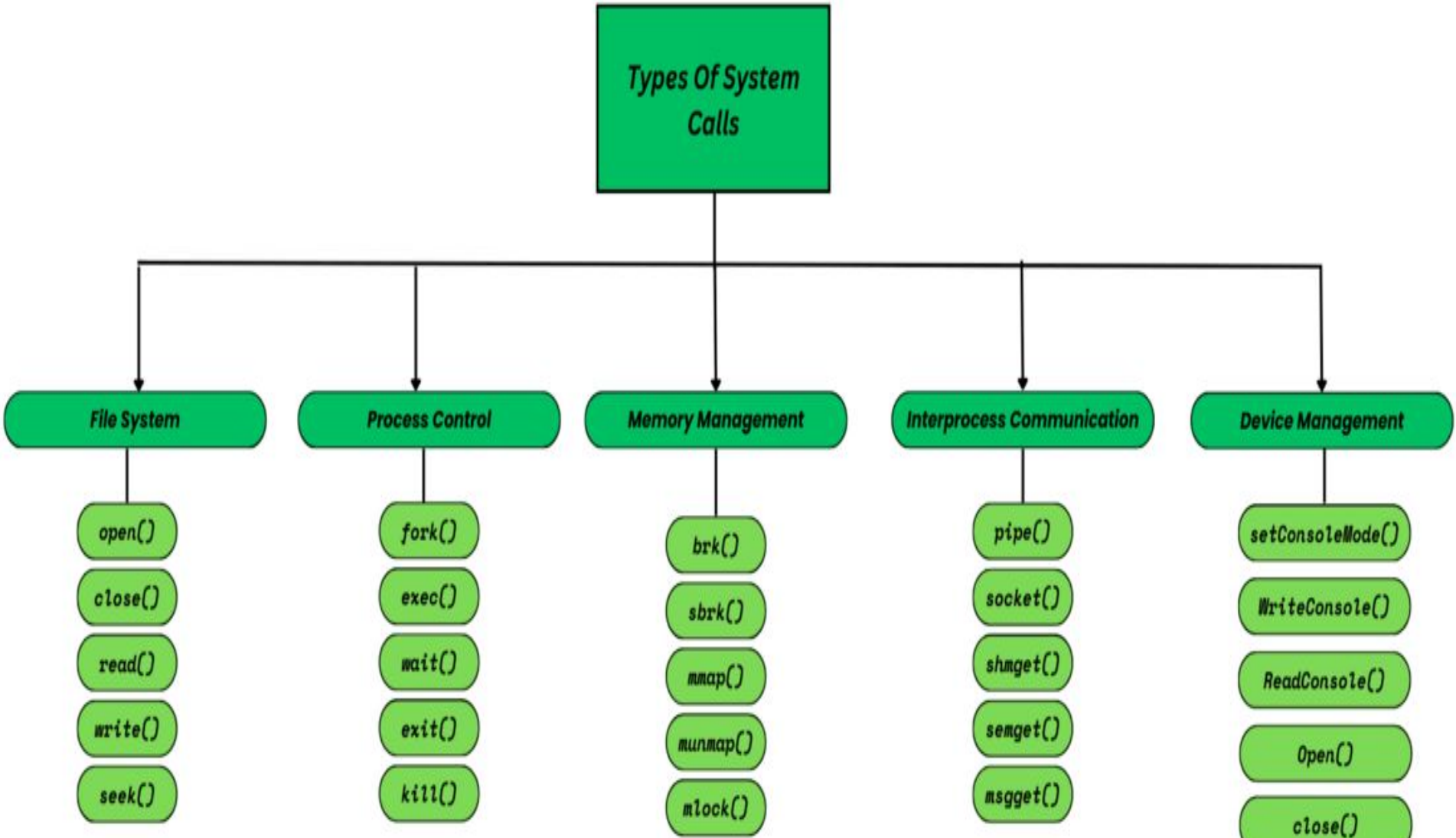
Services provided by an OS are typically related to any kind of operation that a user program can perform like creation, termination, forking, moving, communication, etc. Similar types of operations are grouped into one single system call category. System calls are classified into the following categories:

Purpose of System Calls

System Calls act as a bridge between an operating system (OS) and a running program. They are usually written as assembly language instructions and are detailed in manuals for programmers working with assembly language.

When a program running in user mode needs to access a resource, it makes a System Call. This request is sent to the OS kernel to obtain the needed resource.

System Calls are used for various tasks, such as:

- Creating or executing files in the file system.
- Reading from and writing to files.
- Developing and managing new procedures in programs.
- Making network connections, including sending and receiving data packets.
- Accessing hardware devices like printers and scanners.

System Calls